

FIT3143 PARALLEL COMPUTING · LAB 2 · SEMESTER 2 2026

Lab 2: Prime Search

**Erwynna Soo Wen Xin (36555789) · Taabish Farooq Bhat
(35473932)**



What we set out to answer

Task 1 Can Open MPI beat our Week 4 threaded versions?

Task 2 Do OpenMP threads inside each process help?

Task 3 Does Amdahl's Law predict what we measured?

$n = 130,000,000$ · 7,378,187 primes

Apple M3 Max, 14 cores · everything measured against our Week 4 serial

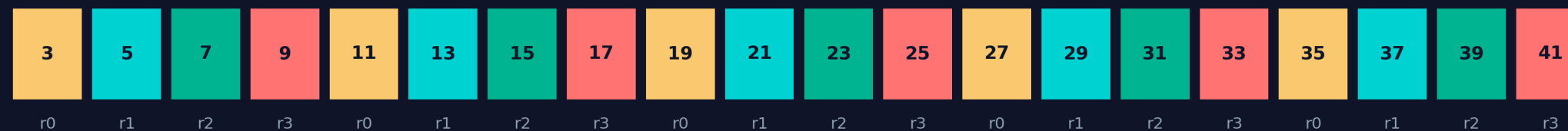
Partitioning Scheme

How the candidates are handed out

Rank r takes $3 + 2r$, then steps by $2p$. Top: $p = 4$, even. Bottom: $p = 3$, one rank idle.

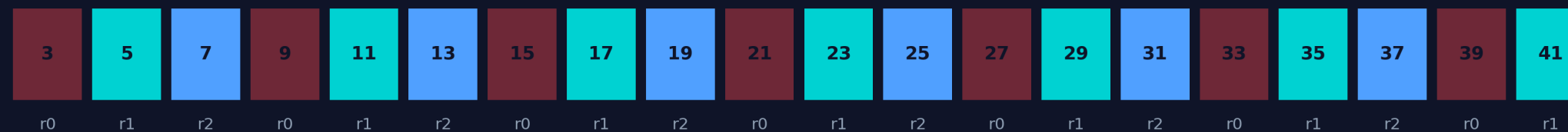
p = 4

no odd factor, so every rank gets a fair mix



p = 3

stride 6, so rank 0 only ever sees multiples of 3



Rank 0 holds 3, 9, 15, 21 ... every one divisible by 3, and is_prime rejects each on its first iteration.

Measured at $n = 130,000,000$: rank 0 finished in 0.012 s having found 2 primes.

Ranks 1 and 2 took 7.6 s each and found 3.69 million each.

In general, for any odd prime d dividing p , one rank in d is idle: $p_{\text{eff}} = p (1 - 1/d)$.

Cyclic stride, no scheduler

Rank r tests $3 + 2r$, then steps by $2p$.

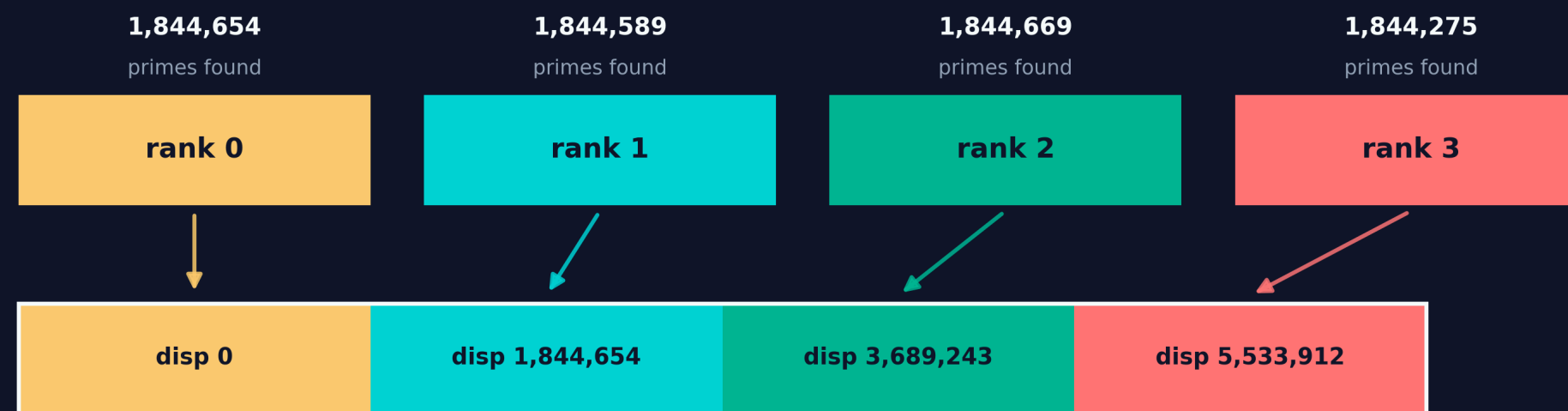
Chosen over a block split because `is_prime(k)` costs about $\sqrt{k}$, so a block hands the last rank every expensive number.

Every rank derives its own range from rank and size. No communication, no runtime scheduling.

Collecting the Results

Bringing the primes back to the root

MPI_Gather of the counts, then MPI_Gatherv places each block at its own displacement.



One contiguous array on the root: 7,378,187 primes

Step 1 MPI_Gather sends one integer per rank, so the root learns the four counts.

Step 2 The root turns the counts into displacements: $\text{disp}[i] = \text{disp}[i-1] + \text{count}[i-1]$

Step 3 MPI_Gatherv writes every block straight into its own slice, in one call.

The blocks arrive interleaved rather than sorted, because the split was cyclic, so the root then qsorts.

Measured cost of the whole exchange: 0.005 s at $p = 1$, 0.036 s at $p = 28$. A shared memory copy, not network traffic.

Gather, sort, write

MPI_Gather of the counts first, so the root learns how many primes each rank found.

The root builds the displacement array, then MPI_Gatherv lands each rank's block at its own offset in one call.

Root qsorts, because a cyclic split arrives interleaved, then writes.

Output is byte-identical to the Week 4 serial reference, verified by diff.

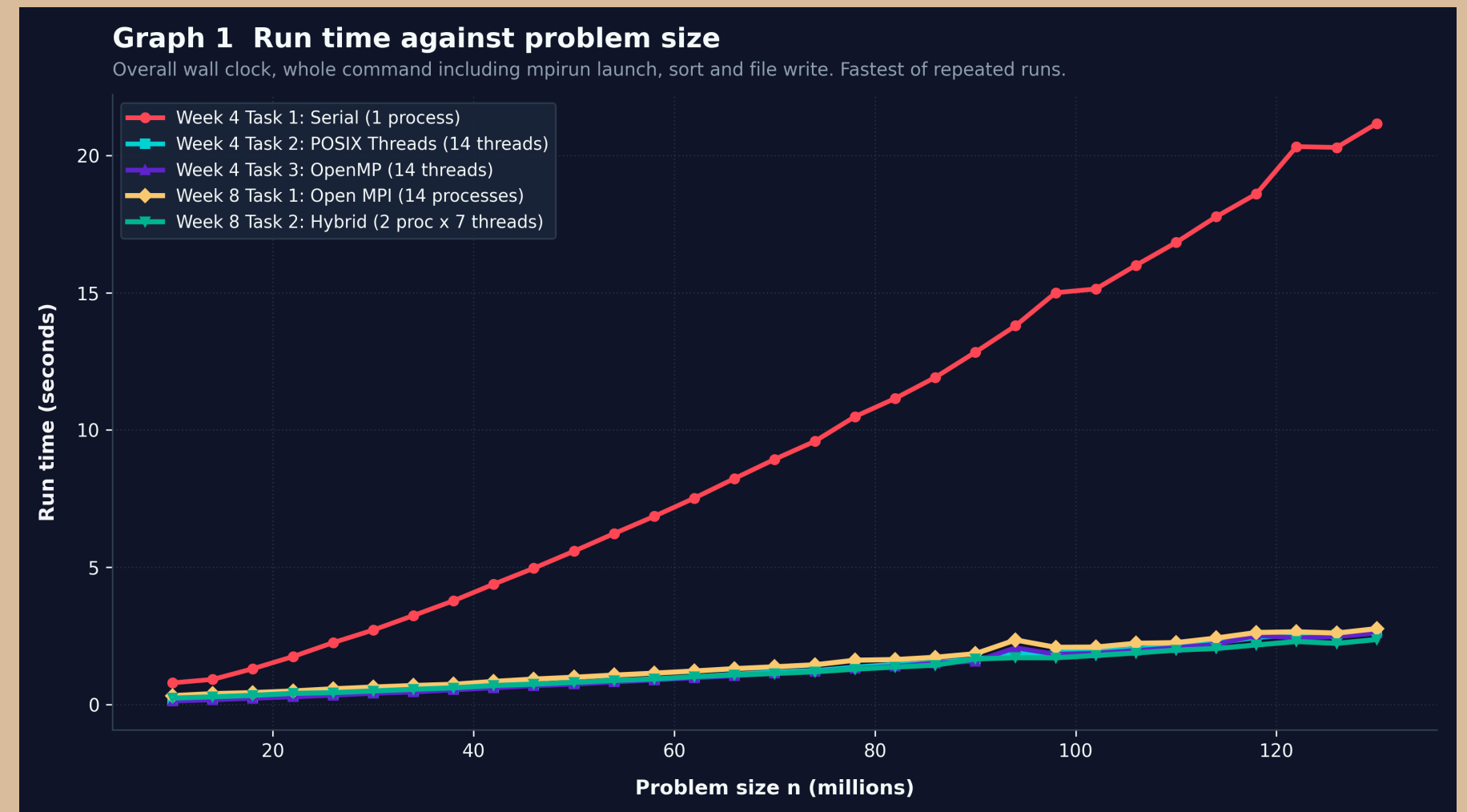
Graph 1

Run time vs problem size

31 values of n, 10M to 130M, at 14 workers.

Serial 21.3 s → Open MPI 3.2 s → OpenMP 3.1 s

Open MPI sits just above OpenMP at every n. The gap is a constant, not a slope: 0.184 s of mpirun launch and MPI_Init, measured by paired sampling.



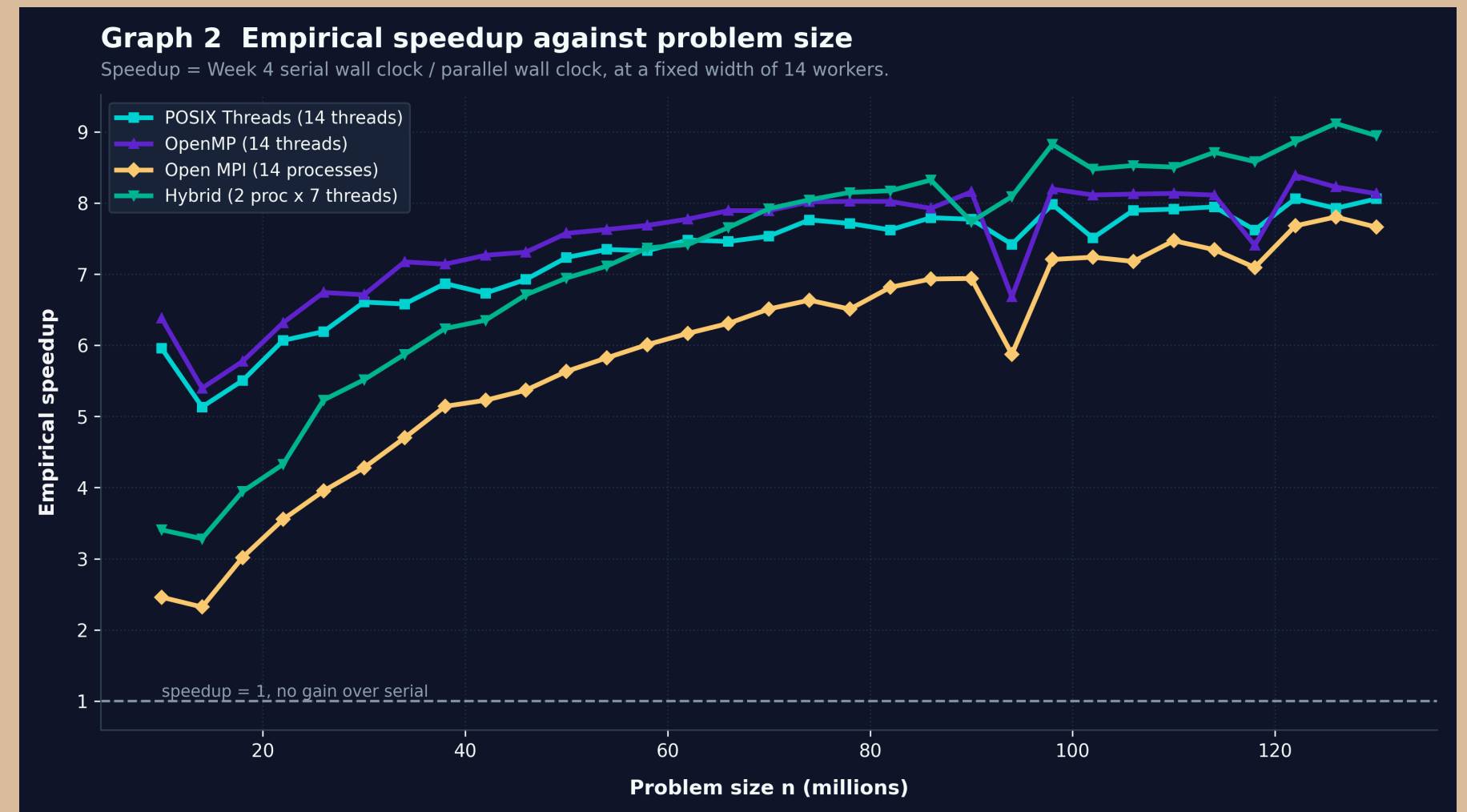
Graph 2

Speedup vs problem size

Against the Week 4 serial, at 14 workers.

3x at $n = 10\text{M}$ rising to 7.8x at $n = 126\text{M}$, still climbing. Speedup improves with n because the launch cost is fixed while the useful work grows.

Our own caveat: the Week 8 search uses a $6\text{k} \pm 1$ test against Week 4's $d \pm 2$, worth about 1.4x before any parallelism.



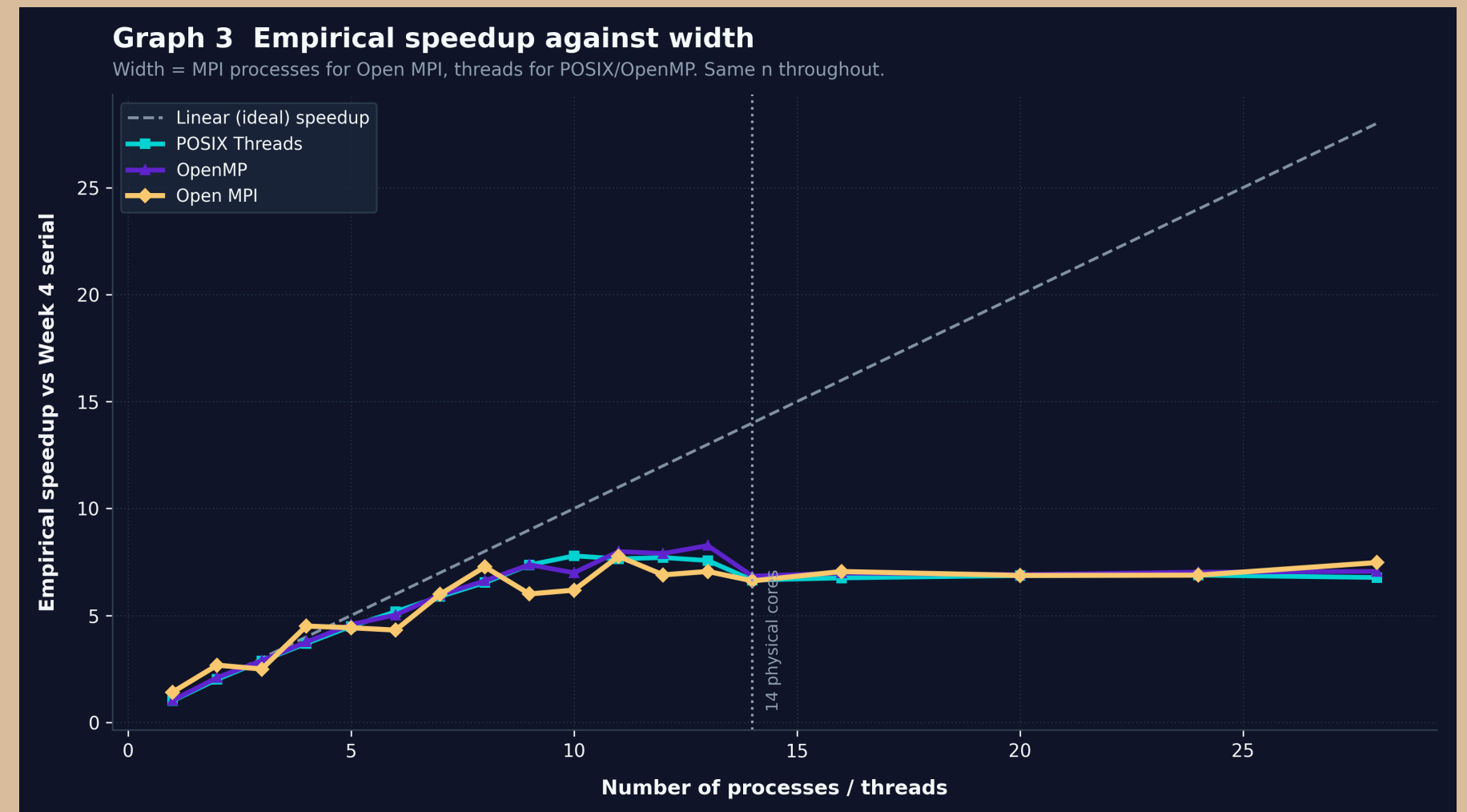
Graph 3

Speedup vs width

Width 1 to 28, twice the core count, at $n = 130M$.

All three plateau near 7x, not 14x. The M3 Max has 10 performance cores plus 4 much slower efficiency cores, and past 14 there are more workers than cores.

Open MPI alone shows a saw tooth. $p = 3$ is slower than $p = 2$. We come back to that in Task 3.



Task 2: The Hybrid Design

Two levels of parallelism in Task 2

MPI strides across processes. OpenMP hands out chunks dynamically inside each one.



`schedule(dynamic, 1000)` a thread that draws a cheap chunk comes straight back for another, so threads self balance at runtime in a way the fixed MPI stride cannot.

Each thread writes its own pre-allocated buffer, so there is no lock and no false sharing on the hot path.

Only the process count is congruence bound. That is why 2 x 7 (2.29 s) beats 7 x 3 (2.50 s) at the same width.

MPI across, OpenMP within

MPI_Init_thread at `MPI_THREAD_FUNNELED`, because only the main thread calls MPI.

The same 2p stride picks each process's share; `schedule(dynamic, 1000)` then splits it across threads.

Each thread writes its own pre-allocated buffer, so no lock and no false sharing on the hot path.

The merge into one per-rank array is the one phase that grows with thread count.

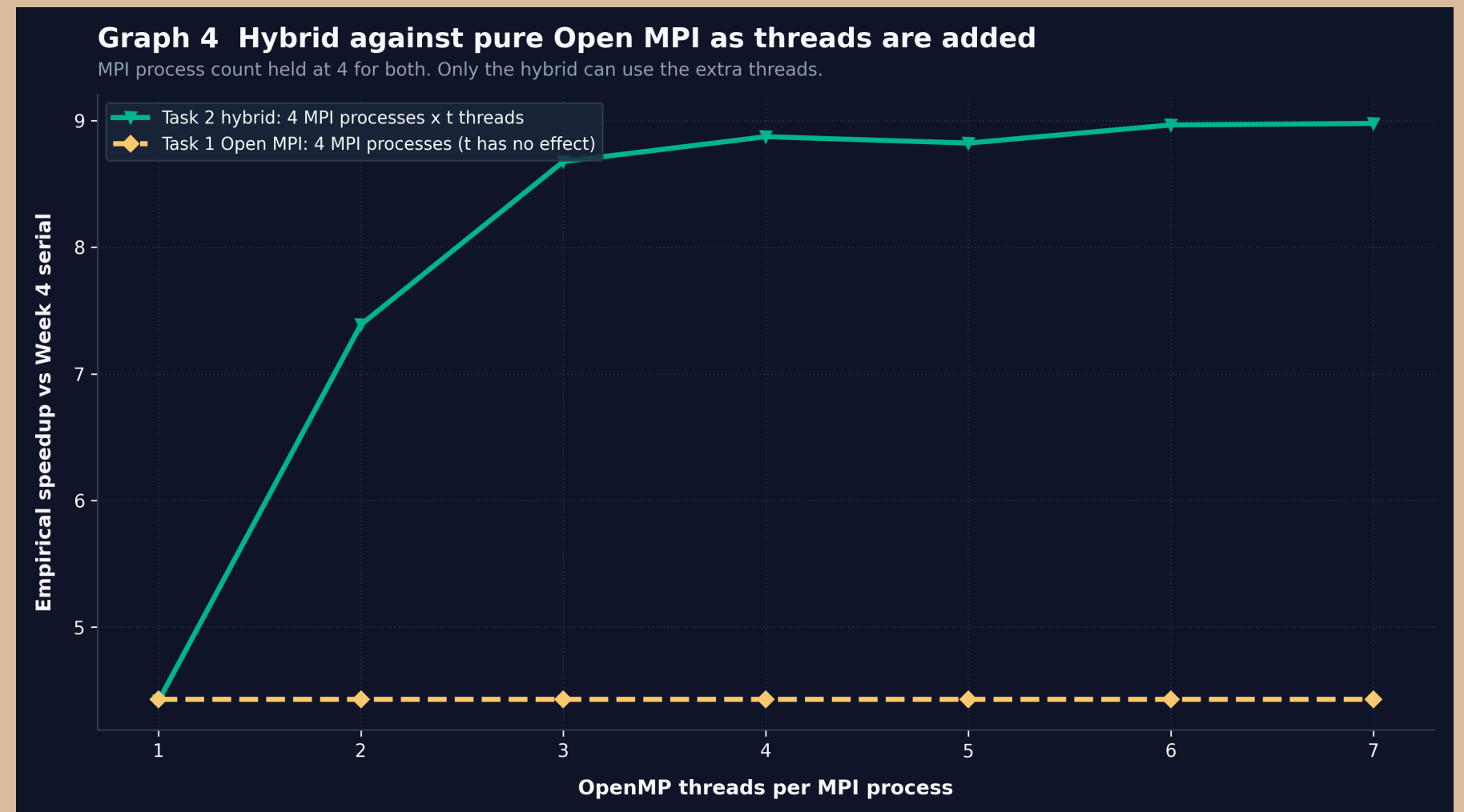
Graph 4

Hybrid vs pure MPI

MPI processes held at 4 for both lines. Only the hybrid can use the extra threads.

Pure Open MPI is flat at 4.80 s. Thread count means nothing to it.

Hybrid falls 4.82 s → 2.37 s across 1 to 7 threads.
Returns flatten past 4: 4 x 4 is already 16 workers on 14 cores.



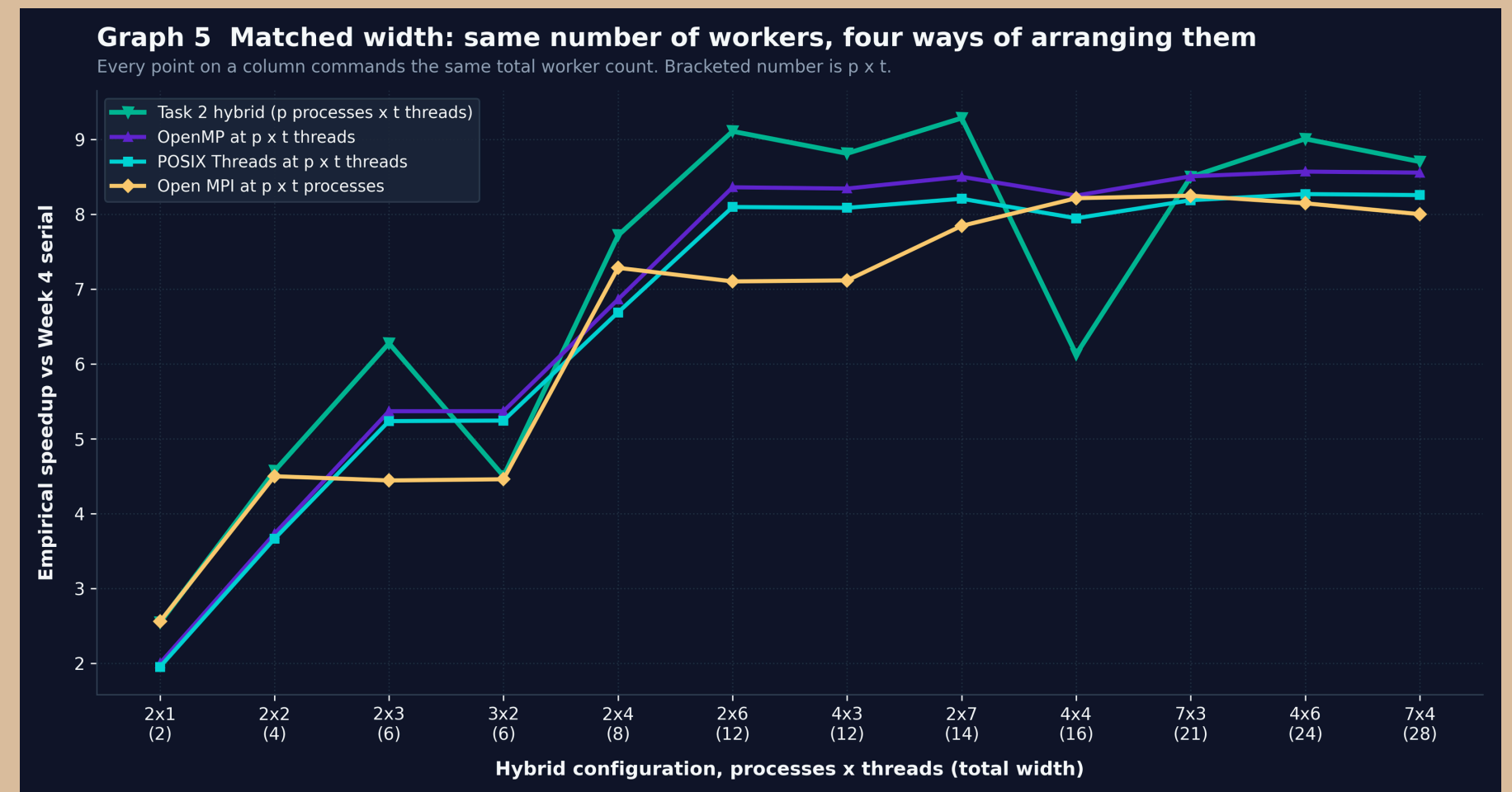
Graph 5

Matched width

Same worker count, four ways of arranging it, as the spec requires: 3 processes x 2 threads is compared against 6 OpenMP threads, not 3.

Best hybrid is 2 x 7 at 2.29 s, beating OpenMP at 14 threads (2.50 s) and pure MPI at 14 processes (2.71 s).

Few processes, many threads wins. Task 3 explains why.





Task 3: How we split serial from parallel

Three instrumented copies: `serial_instr.c`, `task_1_instr.c`, `task_2_instr.c`. Searches untouched, timers added.

Phases timed as the MAX across ranks, because a collective finishes when the slowest rank arrives.

MPI_Barrier at the start, and a second between search and gather. Our first run reported 7.6 s gather times. That was idle ranks waiting inside MPI_Gather, not communication.

After the barrier, real gather is 0.005 s at $p = 1$ and 0.036 s at $p = 28$.

The model, and what it misses

$$S(p) = 1 / (r_s + r_p / p + \text{kappa}(p))$$

$r_p = 0.9708$, $r_s = 0.0288$. A hard ceiling of 34.7x, set by qsort and the 7.4 million line file write alone.

kappa is measured at every p , not assumed away. It reaches only 0.004 of runtime at $p = 28$, because every rank is on one machine and MPI_Gatherv is a memory copy. Communication is not what stops the speedup here.

The baseline is the same code at $p = 1$, not the Week 4 serial, so an algorithmic win is not counted as a parallel win.

Plain Amdahl at p overshoots our measurement. The error is not noise. It has structure.

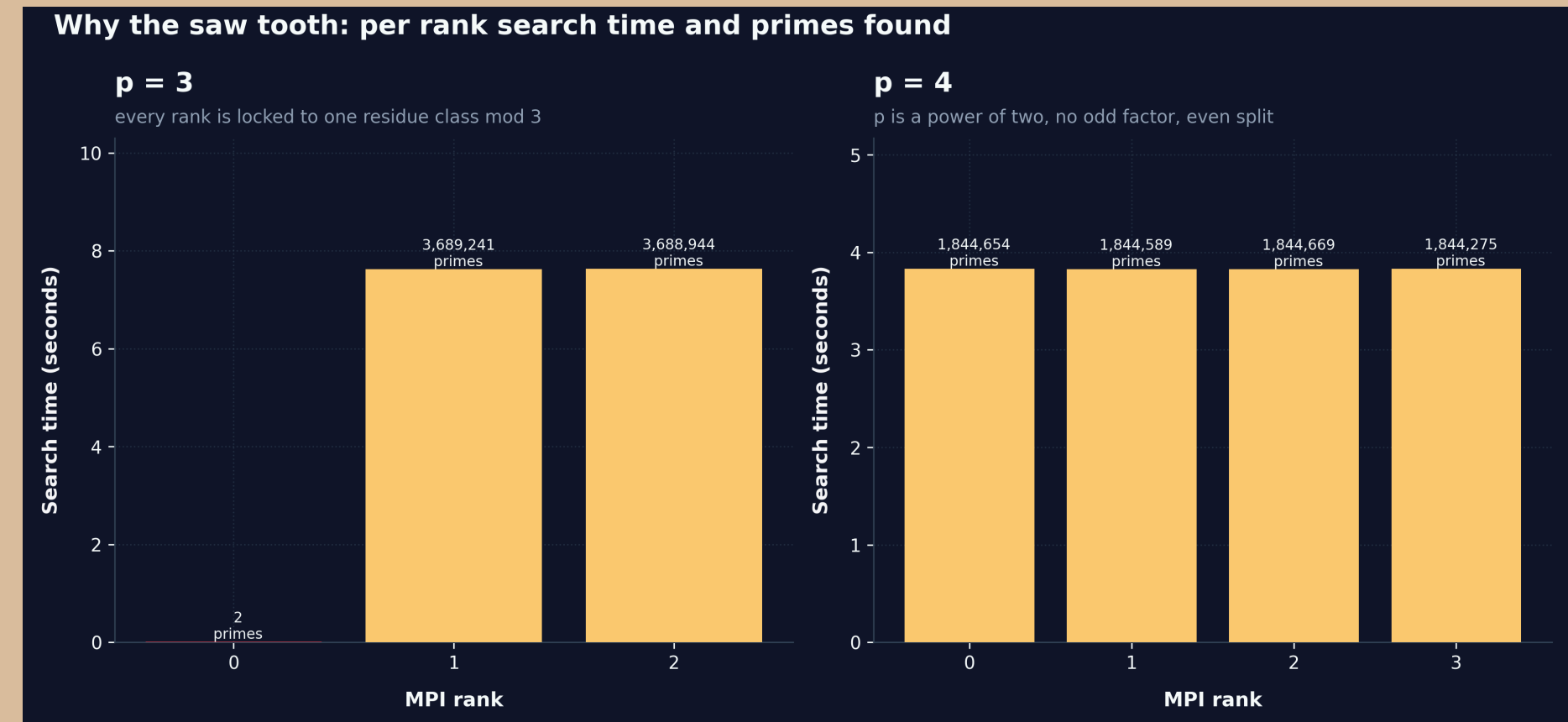
One rank in d does no work

Rank r only ever tests $3 + 2r + 2pk$. For any odd prime d dividing p , that term vanishes mod d , so rank r is locked to one residue class. The rank at $0 \bmod d$ gets only multiples of d , rejected on the first iteration.

$p = 3$: rank 0 found 2 primes in 0.012 s. Ranks 1 and 2 found 3.69 million each in 7.6 s.

$p = 4$: no odd factor, all four ranks within 0.004 s.

$p_{\text{eff}} = p (1 - 1/d)$, and measured imbalance matches $1/(1 - 1/d)$ at every width.



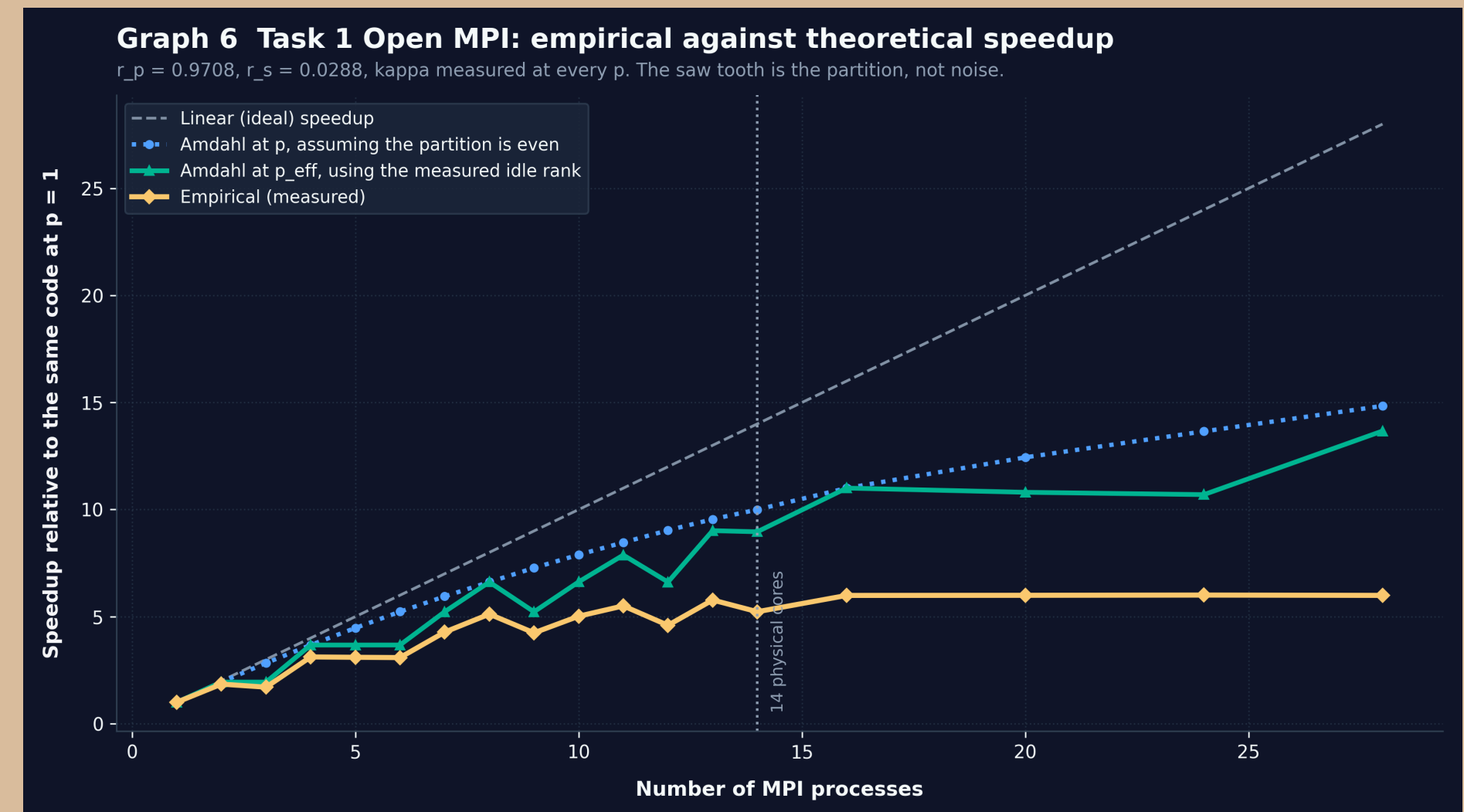
Graph 6

Task 1: theory vs measurement

Three curves: linear ideal, Amdahl at p , and Amdahl at p_{eff} .

Amdahl at p is smooth and overshoots. Amdahl at p_{eff} reproduces the saw tooth dip for dip, which is the evidence that imbalance is the dominant error term rather than noise.

What p_{eff} still does not explain is the plateau past 14: core heterogeneity and oversubscription, both properties of the machine, neither in the model.



Graph 7

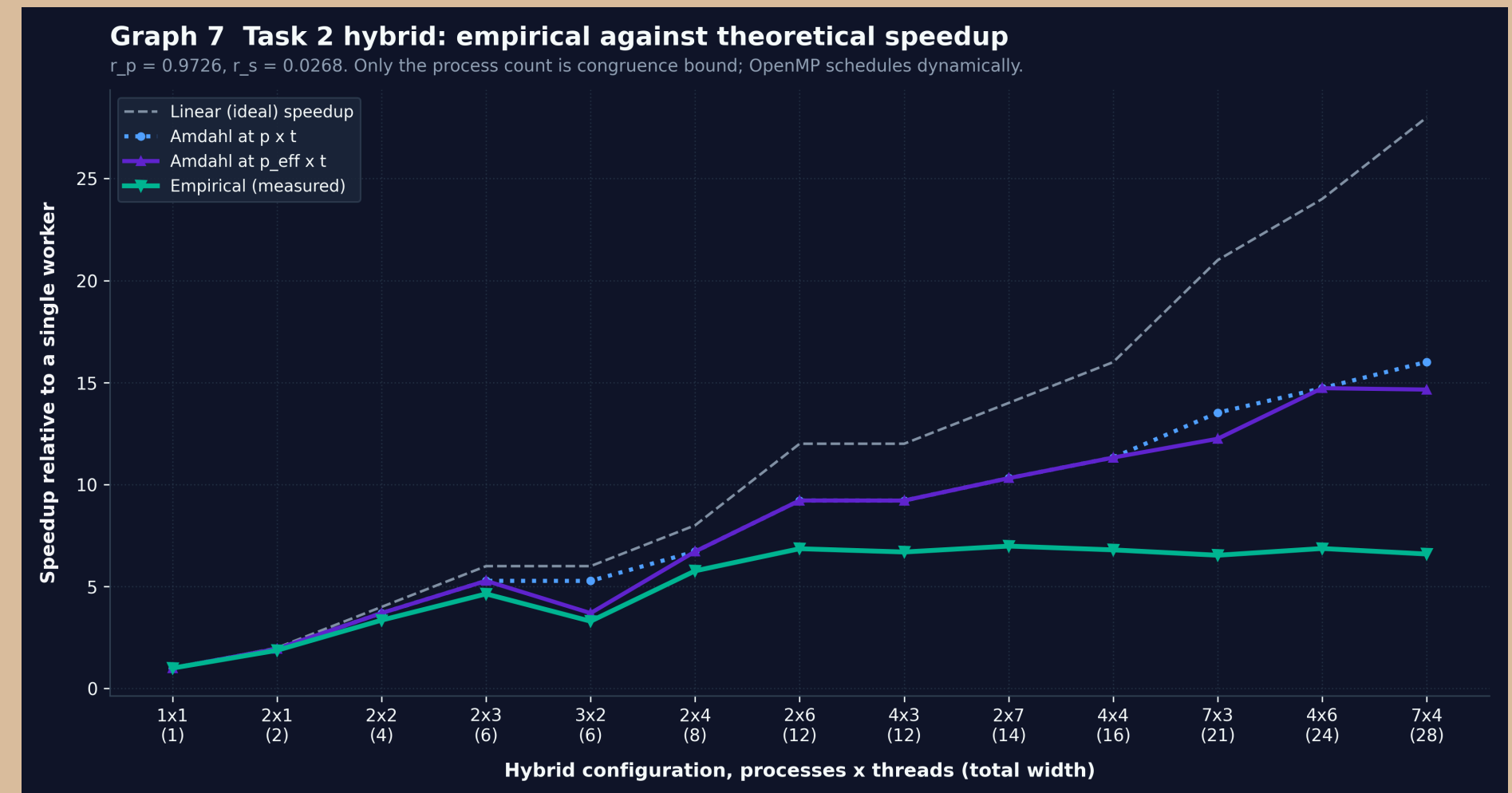
Task 2: theory vs measurement

The hybrid partly escapes the residue problem, because only the MPI stride is congruence bound.

Inside a rank, `schedule(dynamic, 1000)` hands out chunks on demand, so threads self-balance.

2 x 7 has $p_{\text{eff}} = 2$ of 2. 7 x 3 has $p_{\text{eff}} = 6$ of 7. That is why few processes and many threads wins.

$r_p = 0.9726$, $r_s = 0.0268$, close to Task 1 because the serial tail is the same qsort and the same file write.



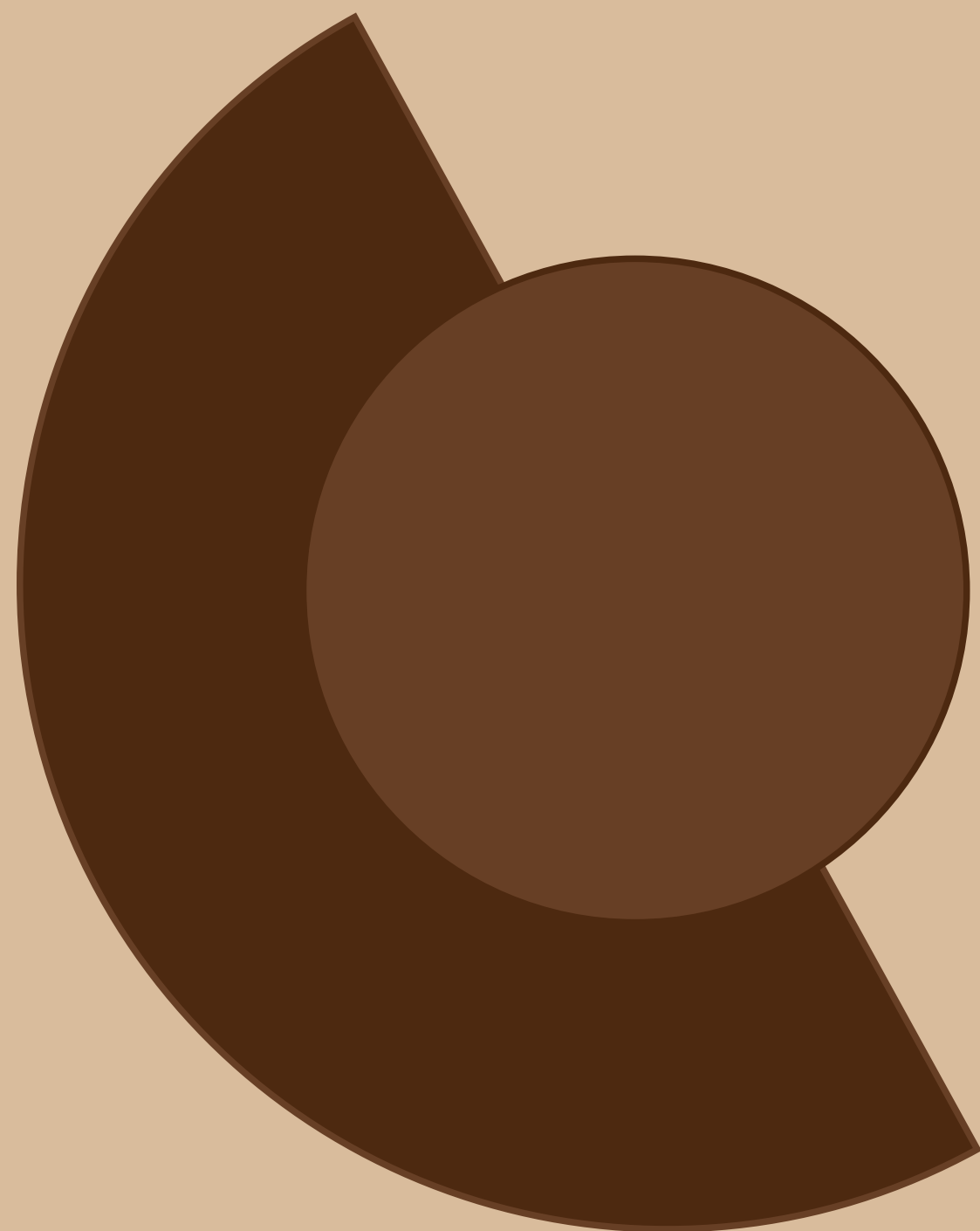
What we found, what we would change

Open MPI matched OpenMP but never beat it, and paid 0.184 s of launch to do it. On one shared memory node, threads are the right tool for this problem.

Communication was negligible. Load imbalance was everything. A measurement that confuses the two is worse than useless.

Break the congruence: interleave chunks of 1000 rather than single strided values, restoring $p_{\text{eff}} = p$. Then overlap the root's file write with the gather to attack r_s .

Single node only, so the missing measurement is a CAAS run across two nodes, where kappa would finally dominate.



Questions

FIT3143 Parallel Computing · Lab 2 · Thank you

ERWYNA SOO WEN XIN · 36555789

esoo0013@student.monash.edu

TAABISH FAROOQ BHAT · 35473932

ttaa0006@student.monash.edu

SUBMITTED FILES

task_1.c · task_2.c · Task3_Performance_Evaluation ·
AI_Declaration

Appendix

Not part of the 7 minute presentation

- A1 Where the time goes, measured phase breakdown
- A2 Full Amdahl parameter table
- A3 Reproducing our results
- A4 Generative AI declaration

A1 Where the time actually goes

Measured phase split of Task 1 across the whole width sweep, from task_1_instr.c.

The search band shrinks as processes are added. The qsort band, the file write band and the imbalance band do not. That is Amdahl's Law drawn from our own measurements rather than asserted.

Communication is barely visible at this scale, which is the point.

[image slot — graph8_phase_breakdown.png]

A2 Full Amdahl parameter table

Task 1: $r_p = 0.9708$ $r_s = 0.0288$ $T(1) = 14.26$ s Task 2: $r_p = 0.9726$ $r_s = 0.0268$ $T(1 \text{ worker}) = 15.16$ s

p	d	p_eff	kappa(p)	S Amdahl	S at p_eff	S measured
1	2^k	1.0	0.00036	1.00	1.00	1.00
2	2^k	2.0	0.00045	1.94	1.94	1.85
3	3	2.0	0.00059	2.83	1.94	1.71
4	2^k	4.0	0.00069	3.67	3.67	3.12
5	5	4.0	0.00069	4.47	3.67	3.10
6	3	4.0	0.00059	5.23	3.67	3.09
7	7	6.0	0.00068	5.95	5.23	4.28
8	2^k	8.0	0.00097	6.62	6.62	5.12
9	3	6.0	0.00079	7.27	5.22	4.24
10	5	8.0	0.00083	7.89	6.62	5.02
11	11	10.0	0.00107	8.46	7.87	5.50
12	3	8.0	0.00099	9.03	6.61	4.59
13	13	12.0	0.00121	9.55	9.01	5.78
14	7	12.0	0.00184	10.00	8.96	5.23
16	2^k	16.0	0.00138	11.00	11.00	5.99
20	5	16.0	0.00303	12.44	10.81	6.00
24	3	16.0	0.00397	13.65	10.70	6.01

d is the smallest odd prime factor of p.

p_eff = p (1 - 1/d), the number of ranks doing real work.

kappa is measured at every p, never assumed.

Every figure here is generated by make_graphs.py straight from the CSVs, so the table and the charts cannot disagree.

Full table, both tasks:
amdahl_summary.csv

A3 Reproducing our results

Nothing in this deck is hand copied. Every number and every chart regenerates from the repository:

```
./run_benchmarks.sh      results_by_n.csv, results_by_procs.csv, results_hybrid_t.csv,  
                        results_hybrid_total.csv, plus a diff against the serial reference
```

```
python3 run_phases.py    phases_task1.csv, phases_task2.csv, rank_balance.csv
```

```
python3 measure_launch.py launch_overhead.txt
```

```
python3 make_graphs.py   graphs 1 to 9, and amdahl_summary.csv
```

```
python3 make_diagrams.py the three explanatory diagrams
```

```
python3 make_task3_doc.py Task3_Performance_Evaluation.md
```

The sweep is parameterised, so the same script runs a reduced version on CAAS without editing anything:

```
N_MAX=40000000 WIDTHS="1 2 4 8 16" REPS=1 ./run_benchmarks.sh
```

A4 Generative AI declaration

Declared under item 9 of the assessment specification. No AI was used during the presentation or the Q and A, per item 10.
Tool: Claude (Anthropic), 1 to 8 September 2026.

Tasks 1 and 2 Review of Taabish's drafts. It found one real defect: task_2.c called memcpy without including <string.h>, so the file did not compile. Nothing in the search, the partitioning or the collectives was written by AI.

Task 3 Help designing the measurement experiment, drafting run_benchmarks.sh, run_phases.py and make_graphs.py, and checking the Amdahl algebra. The measurement design, the choice of baseline and every result are our own, produced by running our own code.

Task 4 Slide text drafting and a check of the deck against the Lab 2 marking rubric.
All graphs were generated from our own measured CSV data.

Full prompt records are attached separately as AI_Prompt_Records.pdf, and the written declaration as AI_Declaration.

We understand and can explain all of the submitted work.